

Welcome to

9. Program Building Blocks

EK Software Security

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
9-program-building-blocks.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Goals for today



Todays goals:

- Talk about the Design Patterns concept in a broader sense
- Present some decisions concerning security and running applications securely
- Patterns can also be found in other areas, like Patterns in Network Architecture

Photo by Thomas Galler on Unsplash

Plan for today



Subjects

- Common constructs
- Recurring code patterns
- Example programs with flaws: OpenSSH, OpenSSL, Windows MS-RPC DCOM, Linux teardrop

Exercises

- Static analysis with PMD
- SBOM Scanning
- Use a XML library in Python

Time schedule



Plan for the day

12:45 - 13:45 – Session 0: Teaching: Code Patterns and Running Software Securely
13:45 - 14:00 – 15min break
14:00 - 14:45 – Session 1: Exercises: PMD Static Analysis and SBOM Scanning
14:45 - 15:00 – 15min break
15:00 - 15:45 – Session 2: Teaching: Sample Infrastructure, API Versioning and Containers
15:45 - 16:00 – 15min break
16:00 - 16:45 – Session 3: Exercises: XML Library in Python

12:45 - 16:20 I will be here at EK

Session 0: Teaching: Code Patterns and Running Software Securely



Design Patterns



- *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides
- The book describes 23 classic software design patterns
- https://en.wikipedia.org/wiki/Design_Patterns
- Ideas of patterns precede this book, but became a more popular subject

Common constructs



- Programs exhibit the same patterns, some examples:
- Solve problems in the same domains
- Need to store lists of strings / characters etc.
- Data structures becomes useful in other programs
- Sorting routines needed in many programs

Auditing code patterns



Various pitfalls, and areas

- **Variable Relationship**
- Examples presented are C buffers
- A variable-pointer to the buffer and the variable with the length
- Their relationship can easily get invalidated, leading to vulnerabilities
- Question: would object oriented programming help?

Structure and Object Mismanagement



- Structures in C can help group related data elements
- Both auditors and attackers can benefit ...
- Object oriented programs and languages encapsulate this
- Responsibility is on the object implementation, good!
- We saw structures last time

Uninitialized Variables



```
Packet *p = SCMalloc(SIZE_OF_PACKET);
if (unlikely(p == NULL)) {
     return 0;
}
ThreadVars tv;
DecodeThreadVars dtv;

memset(&dtv, 0, sizeof(DecodeThreadVars));
memset(&tv,  0, sizeof(ThreadVars));
memset(p, 0, SIZE_OF_PACKET);
```

- Always make sure variables have well defined values.
- Defensive program above use memset to clear contents of buffers
- Example from Suricata allocating memory, checking it got the memory, clearing contents

Manipulating Lists - and other data structures



- Storing data in a list is nice, can add and remove elements
- Single linked list, start from head and go through list ... slow
- Double linked list can go back again from element
- More work updating, moving more pointers ... complex, may introduce errors
- Example data structure double linked list
  https://algorithms.tutorialhorizon.com/doubly-linked-list-complete-implementation/
- Multiple problems can arise, also using the wrong structure for something can result in vulnerabilities

Dont write your data structure libraries yourself

Linux Teardrop



```
#! /usr/bin/env python3
if attack == '0':
  print "Using attack 0"
  size=36    offset=3   load1="\x00"*size
  i=IP()   i.dst=target   i.flags="MF"   i.proto=17

  size=4     offset=18  load2="\x00"*size
  j=IP()   j.dst=target   j.flags=0   j.proto=17   j.frag=offset
  send(i/load1)
  send(j/load2)
```

- IP fragments, packets are split when crossing a link with lower MTU
- If fragments created by an attacker are overlapping it creates a problem
- Scapy example code by Sam Bowne, full code can be found at: https://samsclass.info/123/proj10/teardrop.htm
- what is a Maximum Transmission Unit (MTU)
  See https://en.wikipedia.org/wiki/Maximum_transmission_unit for a description, related/similar attack setting source=destination https://en.wikipedia.org/wiki/LAND

Other problems



- Hashing algorithms
- There have been multiple problems with hashing algorithms
- Denial of Service and arbitrary code can be the result
- Example vulns from popular programming languages, others have similar!
  https://www.cvedetails.com/vulnerability-list/vendor_id-74/product_id-128/cvssscoremin-7/cvssscoremax-7.99/PHP-P
  html search for hash
  https://github.com/bk2204/php-hash-dos specific example in PHP 7.0.0 rc3-3
  https://www.ruby-lang.org/en/security/ Ruby has some
- also when programmers selected wrong, or weak, hashing algorithms for passwords

Control Flow



- If constructs, make sure to exercise each path
- Case/switch constructs, make sure to catch a default
- Loops being subverted to create buffer overflow, terminating conditions
- Forgetting a `break` in a case
- Picture selected due to the Apple "goto fail- certificate validation https://dwheeler.com/essays/apple-goto-fail.html

Apple Goto Fail CVE-2014-1266



```
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail;  /* MISTAKE THIS LINE SHOULD NOT BE HERE */
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;
```

- Always going to goto fail, second goto is always active
- Leading to later checks not performed
- Example code from*Anatomy of a "goto fail" – Apple's SSL bug explained*

https://nakedsecurity.sophos.com/2014/02/24/anatomy-of-a-goto-fail-apples-ssl-bug-explained-plus-an-unofficial-patch/

Loop running over buffer



- MS-RPC DCOM buffer overflow
- Ended up in the Blaster worm infecting : https://en.wikipedia.org/wiki/Blaster_(computer_worm)
- notice the timeline, even if patches ARE available people didn't patch
- This is why we have *patch tuesdays*
- Blaster was not as fast as SQL Slammer, which infected the internet in just minutes
- Other example from book, NTPD, which is also a common and thought to be safe service to run
- And `mod_php` nonterminating buffer vulnerability
- Plus a few off-by-one errors in `mod_rewrite` and OpenBSD ftpd
- ...

Who do you trust?



- Example programs with flaws: OpenSSH, OpenSSL, Microsoft Windows, Linux
- Who do you trust?
- Can we trust any software?
- What about Suricata? - CVE list

  https://www.cvedetails.com/vulnerability-list/vendor_id-13364/product_id-27771/Openinfosecfoundation-Suricata.html
- Spoiler: Bypass vulnerabilities, getting data past the IDS, Denial of Service crashing the IDS
- Trust is also a design decision – more on supply chain later

Suricata IDS – older CVEs



- CVE-2013-5919 Suricata $<$ 1.4.6, CVSS 5.0: crash on malformed SSL record
  https://www.cvedetails.com/cve/CVE-2013-5919/
- CVE-2014-6603 Suricata $<$ 2.0.4, CVSS 5.0: crafted SSH banner, rule bypass, large allocation or OOB write
  https://www.cvedetails.com/cve/CVE-2014-6603/
- CVE-2015-0971 Suricata $<$ 2.0.8, CVSS 5.0: DER parser crash via SSL/TLS certificates
  https://www.cvedetails.com/cve/CVE-2015-0971/
- CVE-2015-8954 Suricata $<$ 2.0.6, CVSS 9.8: `MemcmpLowercase` skips first byte, IPS bypass via crafted HTTP
  https://www.cvedetails.com/cve/CVE-2015-8954/
- CVE-2017-7177 Suricata $<$ 3.2.1, CVSS 7.5: IPv4 defrag evasion, IP protocol not checked when matching fragments
  https://www.cvedetails.com/cve/CVE-2017-7177/
- CVE-2017-15377 Suricata $<$ 4.x, CVSS 7.5: redundant content checks until recursion limit (3000), DoS
  https://www.cvedetails.com/cve/CVE-2017-15377/

Suricata IDS – selected CVEs



- CVE-2018-6794 Suricata $<$ 4.0.4, CVSS 5.3: TCP data before handshake, HTTP response signatures bypassed (Snort 2.9.4 too)
  https://www.exploit-db.com/exploits/44247
- CVE-2024-32664 Suricata, CVSS 5.3: base64 out-of-bounds write via crafted traffic or datasets
  https://github.com/OISF/suricata/security/advisories/GHSA-79vh-hpwq-3jh7
- CVE-2024-37151 Suricata, CVSS 7.5: fragments reusing IP ID break reassembly, policy bypass
  https://github.com/OISF/suricata/security/advisories/GHSA-qrp7-g66m-px24
- CVE-2025-59147 Suricata, CVSS 7.5: crafted multiple SYNs, TCP detection bypass
  https://bugzilla.redhat.com/show_bug.cgi?id=2400928
- CVE-2025-64330 Suricata, CVSS 7.5: heap over-read logging verdict in eve.alert (non-default config)
  https://github.com/OISF/suricata/security/advisories/GHSA-83v7-gm34-f437

An IDS must parse hostile traffic – and agree with the endpoint on how it should be understood

Resources



Microsoft Security Development Lifecycle (SDL) – The SDL is a software development process from Microsoft that helps developers build more secure software. It helps you address security compliance requirements while reducing development costs.

Open Web Application Security Project (OWASP) – OWASP is an online community that produces freely available articles, methodologies, documentation, tools, and technologies in the field of web application security.

Pushing Left, Like a Boss – A series of online articles that outlines different types of application security activities that developers should complete to create more secure code.

Microsoft identity platform – The Microsoft identity platform is an evolution of the Azure AD identity service and developer platform. It's a full-featured platform that consists of an authentication service, open-source libraries, application registration and configuration, full developer documentation, code samples, and other developer content. The Microsoft identity platform supports industry-standard protocols like OAuth 2.0 and OpenID Connect.

Security best practices for Azure solutions – A collection of security best practices to use when you design, deploy, and manage cloud solutions by using Azure. This paper is intended to be a resource for IT pros. This might include designers, architects, developers, and testers who build and deploy secure Azure solutions.

Security and Compliance Blueprints on Azure – Azure Security and Compliance Blueprints are resources that can help you build and launch cloud-powered applications that comply with stringent regulations and standards.

Next steps

In the following articles, we recommend security controls and activities that can help you design, develop, and deploy secure applications.

* Design secure applications * Develop secure applications * Deploy secure applications

Source: https://docs.microsoft.com/en-us/azure/security/develop/secure-dev-overview

Architecture for modern-day web applications



The average modern-day web application probably makes use of several of the follow- ing technologies:

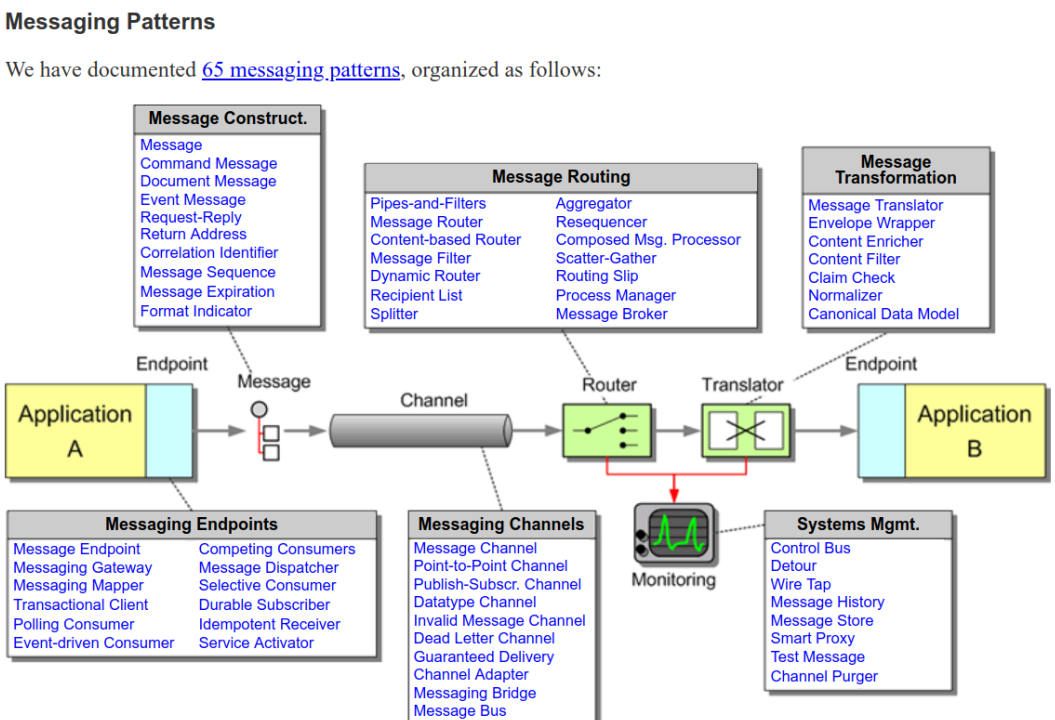
- REST API
- JSON or XML
- JavaScript
- SPA framework (React, Vue, EmberJS, AngularJS)
- An authentication and authorization system
- One or more web servers (typically on a Linux server)
- One or more web server software packages (ExpressJS, Apache, NginX)
- One or more databases (MySQL, MongoDB, etc.)
- A local data store on the client (cookies, web storage, IndexDB)

Source: *Web Application Security*, Andrew Hoffman, 2020, ISBN: 9781492053118

Note: the list could have come from many places, very similar structure

Enterprise Integration Patterns

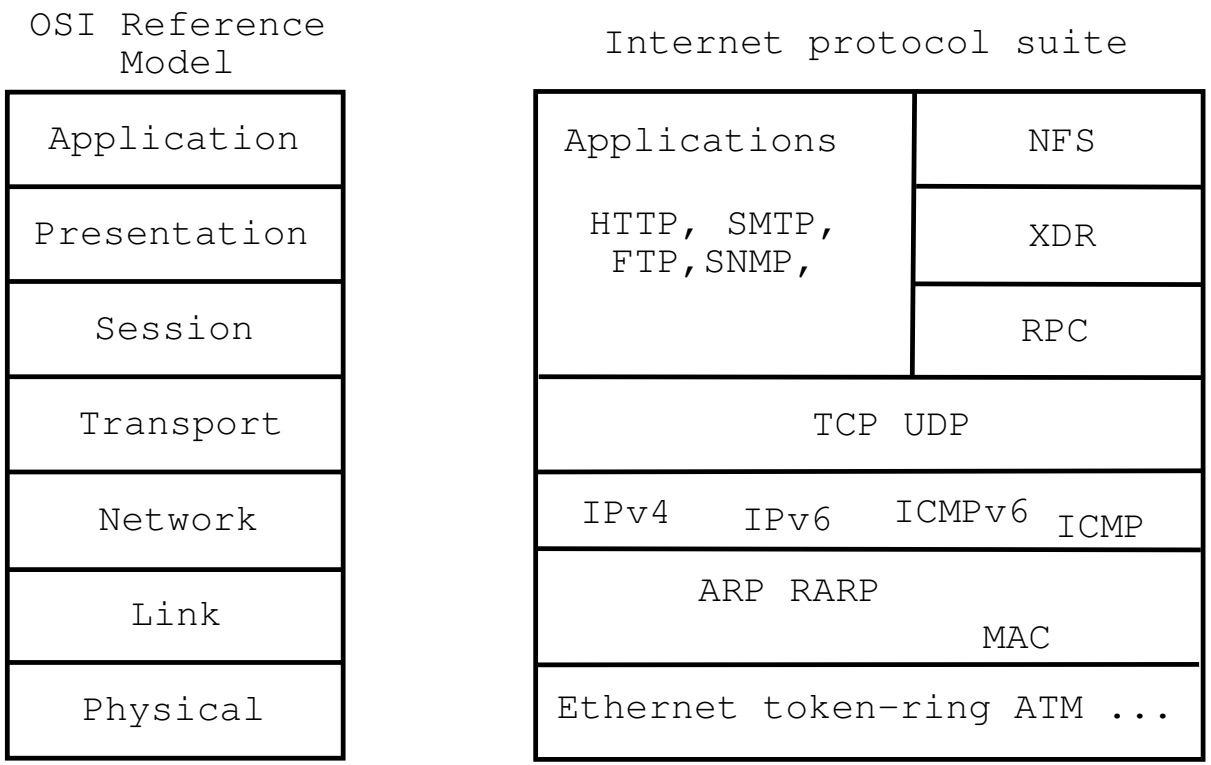




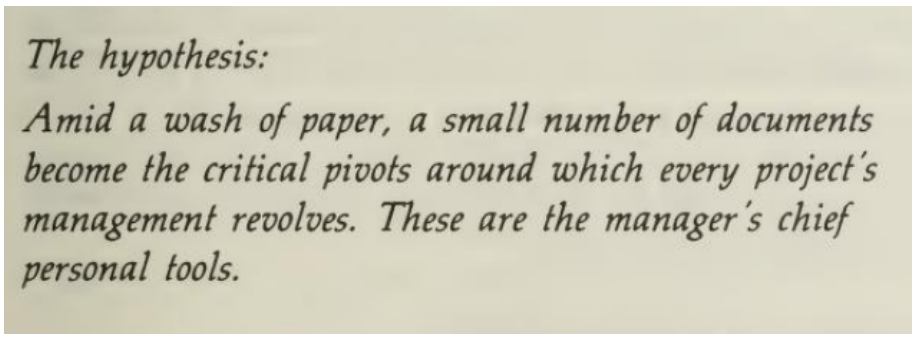
Source: https://www.enterpriseintegrationpatterns.com/ Use patterns when building systems!

OSI Model and Internet Protocols





Document or it didn't happen

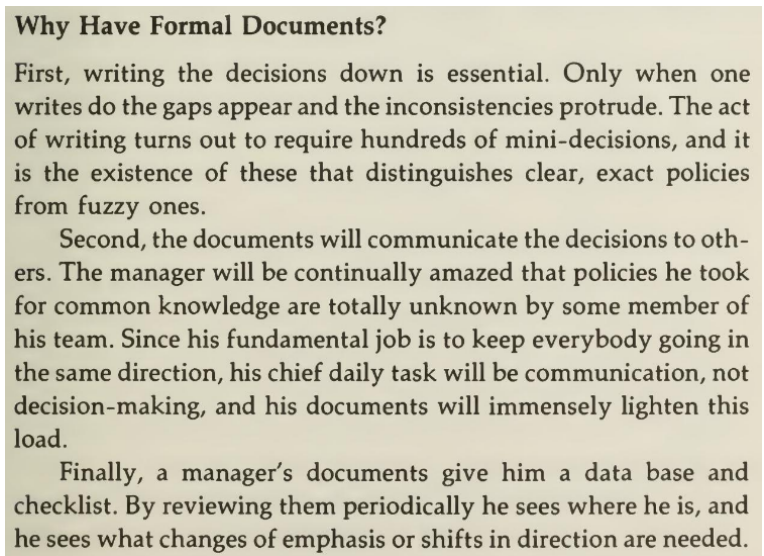


The hypothesis:

Amid a wash of paper, a small number of documents become the critical pivots around which every project's management revolves. These are the manager's chief personal tools.

- Make sure to document!
- There are so many parts, so many decisions, soo many places thing can mess up
- Have a minimum of documentation – and the source files are not enough
- Hint: you can read the Mythical Man Month at https://archive.org/details/MythicalManMonth

Why Have Formal Documents



Why Have Formal Documents?

First, writing the decisions down is essential. Only when one writes do the gaps appear and the inconsistencies protrude. The act of writing turns out to require hundreds of mini-decisions, and it is the existence of these that distinguishes clear, exact policies from fuzzy ones.

Second, the documents will communicate the decisions to others. The manager will be continually amazed that policies he took for common knowledge are totally unknown by some member of his team. Since his fundamental job is to keep everybody going in the same direction, his chief daily task will be communication, not decision-making, and his documents will immensely lighten this load.

Finally, a manager's documents give him a data base and checklist. By reviewing them periodically he sees where he is, and he sees what changes of emphasis or shifts in direction are needed.

Source: *The Mythical Man-Month (Anniversary Edition)* by Frederick P. Brooks Jr.

- I use Git version control for my documentation purposes, some is in Zim Personal Wiki https://zim-wiki.org/
- I also use an IPAM https://spritelink.github.io/NIPAP/

Operating Systems



Computers need software – operating systems, and applications run on an OS:

- OS is the base for your experiments
- Must be managed, installed at least, updated and packages installed on top
- Some applications need a regular OS, others are fine with containers
- Some applications like Python can even use a virtual environment

Select good operating systems, make sure to configure it – **hardened/secure configuration**

First advice use the modern operating systems



Newer versions of Microsoft Windows, Mac OS X and Linux

- Buffer overflow protection
- Stack protection, non-executable stack
- Heap protection, non-executable heap
- *Randomization of parameters* stack gap m.v.

Note: these still have errors and bugs, but are better than older versions

OpenBSD has shown the way in many cases
http://www.openbsd.org/papers/

Always try to make life worse and more costly for attackers

Virtualization

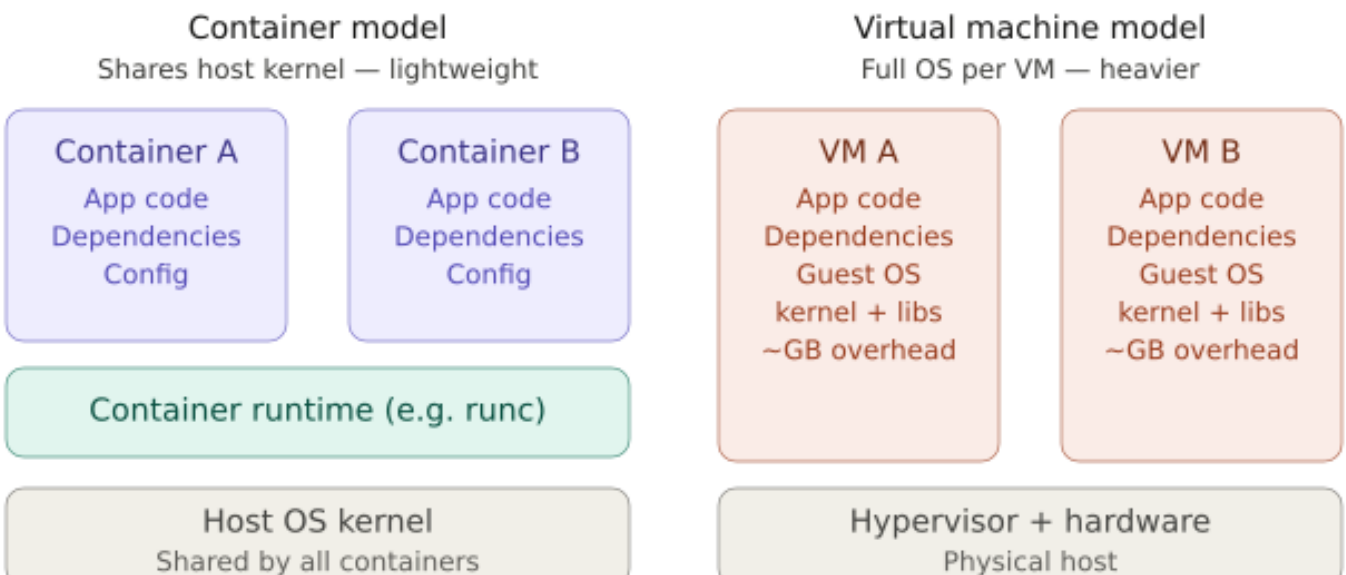


Computers can run multiple operating systems, and OS distributions at the same time:

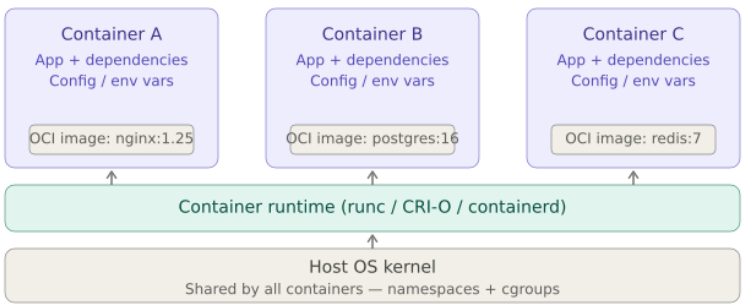
- Full virtual machine virtualization
- Containers – including containers platforms like Kubernetes
- Use it, and create many virtual machines/containers
- Switch context easily, one project - one (or more) VMs
- Create a new VM in seconds – keep one template VM updated that you can clone
- Install crap – delete a VM, retry
- Performance of VMs and containers on modern CPUs is great! Almost like having two physical computers!

Maybe sometime use different physical hosts – one for internal use, one for external use

Virtualization Overview



Containers



- Containers package an application and all its dependencies into a single portable unit
- Share the host OS kernel – lighter and faster to start than full virtual machines
- Docker is the dominant developer-facing tool; images are built from a `Dockerfile` and stored in registries
- Images are not Docker-specific – the industry standard is the **OCI image format**, supported by all modern tools
- **Compose files** allow multi-container applications (e.g. app + database) to be defined and run together
- Pre-packaged systems that require less administration – **only configuration**

Container Standards and Runtimes



- The **Open Container Initiative (OCI)** defines three open standards: image-spec, runtime-spec, and distribution-spec https://opencontainers.org
- Any OCI-compliant image can be built with one tool (e.g. Buildah, Docker) and run with another – no vendor lock-in
- **runc** is the reference low-level runtime implementing the OCI runtime-spec; most runtimes use it under the hood
- Kubernetes communicates with runtimes via the **Container Runtime Interface (CRI)** – a gRPC-based plugin API
- **containerd** and **CRI-O** are the two main CRI-compliant runtimes; Docker's direct integration was removed in Kubernetes v1.24 https://kubernetes.io/docs/setup/production-environment/container-runtimes/
- CRI-O is purpose-built for Kubernetes – lightweight, OCI-compliant, and the default in Red Hat OpenShift https://cri-o.io

Kubernetes



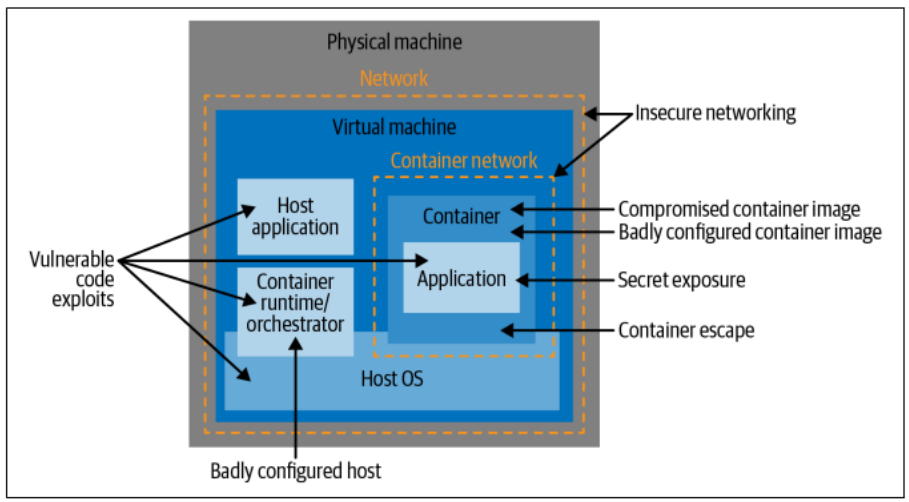
Run K8s Anywhere

Kubernetes is open source giving you the freedom to take advantage of on-premises, hybrid, or public cloud infrastructure, letting you effortlessly move workloads to where it matters to you.

To download Kubernetes, visit the download section.

- Kubernetes (K8s) is an open-source system for automating deployment and management of containers at scale
- Organises containers into *Pods*, grouped by *Deployments*, and exposed via *Services*
- Provides automatic scaling, self-healing, and rolling updates with zero downtime
- Suited to large, complex microservice architectures – significant operational overhead for small projects
- Managed offerings (GKE, EKS, AKS) reduce setup burden but increase cloud dependency

Kubernetes Security is more than just the core



Source: *Container Security*, (CS) Liz Rice, O'Reilly, Apr 2020

- Attacks from inside, attacks inside containers
- Some just use up resources, some attack others, some attack the system from the inside
- *Hacking Kubernetes* has a whole *Chapter 3 Container Runtime Isolation*

Kata Containers – containers in lightweight VMs



Open source container runtime that runs each pod inside its own lightweight virtual machine

Each pod gets its own guest kernel instead of sharing the host kernel with all other containers

For Kubernetes and the user it still looks and behaves like normal containers

Uses a hypervisor like QEMU, Cloud Hypervisor or Firecracker to start the VM

An agent inside the VM, *kata-agent*, starts and manages the containers on request from the runtime on the host

Container files are shared from the host into the VM with virtio-fs

Confidential Containers: the VM runs with encrypted memory (Intel TDX, AMD SEV-SNP), so even the host operator cannot look inside

Kata Containers 2026 vulnerabilities – host escapes



- CVE-2026-44210 (8.8) – pod annotations inject arguments into virtiofsd, which serves the host root filesystem into the guest. Fixed in 3.31.0
- CVE-2026-47243 (8.7) – guest root to host root escape via virtio-fs in runtime-rs
- CVE-2026-50540 (9.6) – an unvalidated configuration path annotation gives code execution on the host. Fixed in 4.0.0
- Common pattern: pod annotations that are enabled by default are an attack surface, because whoever can create a pod can influence the host side

Kata Containers 2026 vulnerabilities – guest and CoCo



- CVE-2026-24834 – a container user can modify the guest VM filesystem and get root in the guest (Cloud Hypervisor). Fixed in 3.27.0
- CVE-2026-41326 (8.2) – symlinks bypass the CopyFile policy, so the host can overwrite binaries inside the guest, even in CVMs. Fixed in 3.29.0
- CVE-2026-64676 (5.7) – two mem-agent ttRPC methods skip the agent policy check. Fixed in 4.0.0
- CVE-2026-77176 – genpolicy does not validate CreateContainer mount and storage rules well enough, so a malicious host operator can cause information disclosure
- Advisories:
  https://github.com/kata-containers/kata-containers/security/advisories

Summary: Technology Comparison



Technology	Best For	Complexity	Scalability
VPS	Single apps, full OS control	Low	Vertical (manual)
Containers	Portable, consistent deploys	Medium	Horizontal (manual)
Kubernetes	Large-scale microservices	High	Horizontal (auto)
Virtual Hosts	Multi-site on one server	Low	Limited

- Above does not consider if you are a private person running systems for yourself, friends, family
- If you are a company with a few services
- If you are a large company that want to switch over to something privately owned
- Above are in my opinion good options for running your complete infrastructure!

OWASP Kubernetes Security Cheat Sheet: Container Sandboxing



Cgroups and namespaces isolate containers, but the kernel still presents a large attack surface

Recommends an extra sandbox layer for multi-tenant and highly untrusted clusters, with these open source options:

- Kata Containers – stripped-down VMs with a small resource footprint
- gVisor – its own kernel written in Go, sitting between the container and the host kernel
- Firecracker – a very lightweight VM locked down with seccomp, cgroups and namespaces

The OWASP Secure AI Model Ops Cheat Sheet also recommends microVMs, gVisor, Kata Containers or confidential compute for high-sensitivity workloads

Source:

Kubernetes Security Cheat Sheet, OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Kubernetes_Security_Cheat_Sheet.html

Virtual Hosts in Web Servers



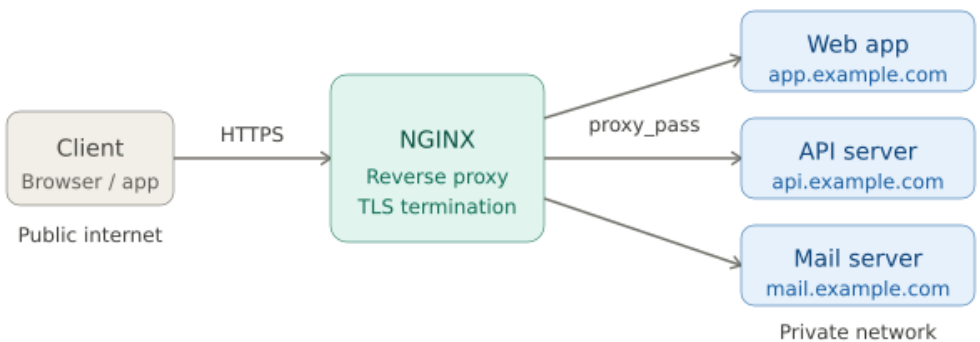
```
server { server_name example.com www.example.com; root /var/www/example.com; }
server { server_name example.org; root /var/www/example.org; }
```

- One server with one IP address can serve many websites
- DNS for both **example.com** and **example.org** points to the same IP
- The browser sends the name it wants in the HTTP **Host** header, the web server picks the matching site
- With HTTPS the name is also sent in TLS as **SNI**, so the server can present the right certificate

Side note: Digital Ocean and NGINX as a reverse proxy

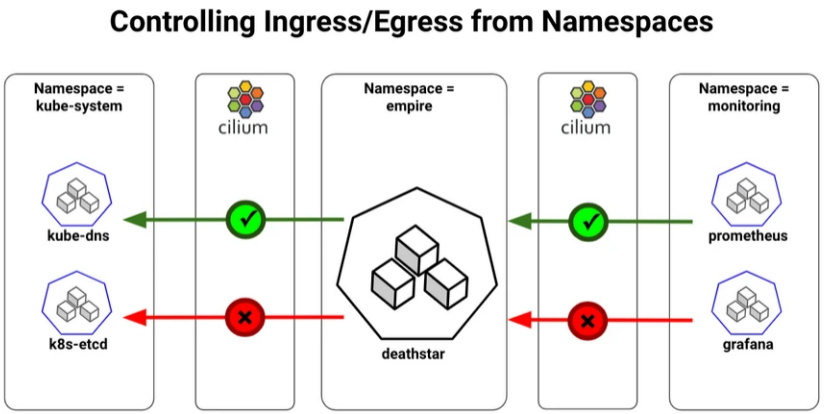
That said DigitalOcean has very good documentation for lots of things! example:
https://www.digitalocean.com/community/tutorials/how-to-configure-nginx-as-a-reverse-proxy-on-ubuntu-22-04

NGINX as a Reverse Proxy



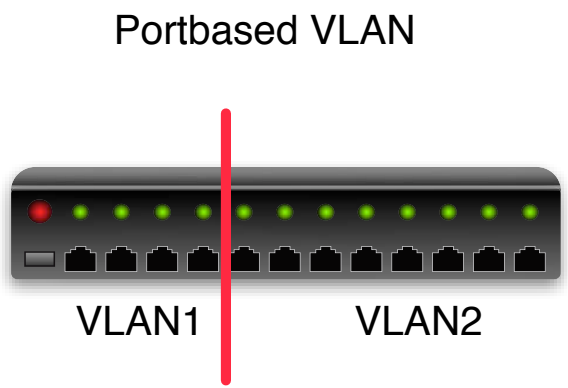
- A reverse proxy sits in front of backend servers, accepting client requests and forwarding them internally
- NGINX uses the `proxy_pass` directive to route traffic to upstream services by hostname or IP
- Can provide TLS termination, load balancing, caching, and request buffering at the edge
  A single system with NGINX get certificates with Let's Encrypt (or alternatives)

Kubernetes Network parts – NetworkPolicy



- Container Network Interface (CNI) and Cloud Native Computing Foundation (CNCF) project https://github.com/containernetworking/cni
- Multiple options – popular and common ones Calico, Flannel and Cilium
- Cilium - I have followed Cilium for a while, so I choose that

Together with Firewalls - Virtual LAN (VLAN)



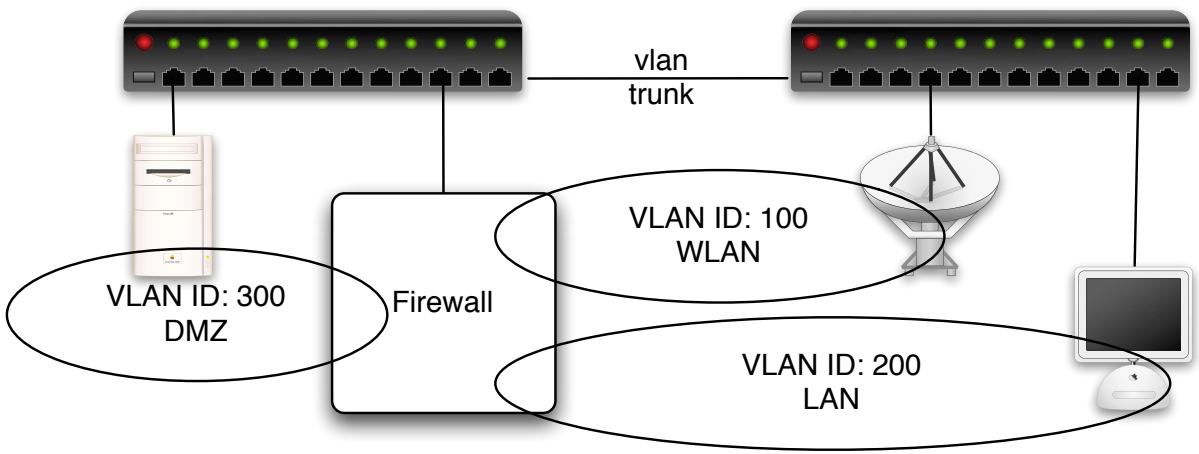
Managed switches often allow splitting into zones called virtual LANs

Most simple version is port based

Like putting ports 1-4 into one LAN and remaining in another LAN

Packets must traverse a router or firewall to cross between VLANs

Virtual LAN (VLAN) IEEE 802.1q



Using IEEE 802.1q VLAN tagging on Ethernet frames

Virtual LAN, to pass from one to another, must use a router/firewall

Allows separation/segmentation and protects traffic from many security issues

Network Segmentation: Which VLANs to create



Using VLANs to segment networks we can extend it to the rest of the network, and an example of isolated segments could be:

- Guest network for cabled and Wi-Fi clients
- Client networks can be isolated per department, floor or building
- Server networks for multiple purposes – development, staging, testing, production, manufacturing, ...
- Internet servers in a demilitarized zone (DMZ) which would make it less likely that compromised internet server would affect the rest of the network
- Dedicated printer network
- **Management network for virtualisation, one for network management, one for storage**

Example, Restricting the Kubernetes API



Watch out when learning, you may expose an *admin interface*

> **By default, the Kubernetes API server listens on port 6443 on the first non-localhost network interface, protected by TLS**. In a typical product ion Kubernetes cluster, the API serves on port 443. The port can be changed with the –secure-port, and the listening IP address with the –bind -address flag.
>
> ...
>
> If your cluster uses a **private certificate authority, you need a copy of that CA certificate configured into your /.kube/config on the cli ent**, so that you can trust the connection and be confident it was not intercepted.

- If an attacker can run Kubectl with credentials – they can do anything
- Defense in Depth is suggested – so restrict at multiple levels
- Only allow connections from specific network – firewall filtering
- Use TLS and verify/copy certificates

Proxy servers and Web Application Firewalls (WAF)



Filtering at higher layers is also possible

- Web proxies for clients can help security a lot – a centralized filter for everyone
- Reverse proxies for web applications are called Web Application Firewalls (WAF) – and filter incoming web requests, and outgoing answers. Can help with attacks like SQL injection and exfiltration of data
- Depending on your network it can replace or be combined with filtering on DNS servers, and I would prefer to filter domains with DNS
- I would also prefer blocking large prefixes of IP destinations using routers/stateless packet filters – maybe use BGP for distributing *lists* or automation for downloading block lists

Also great home lab projects!

CrowdSec – Crowd sourced security information



Detect and block Log4j exploitation attempts with CrowdSec

If you work in Infosec, you had a very lousy weekend. And that's because of the Log4j zero-day vulnerability (CVE-2021-44228) that was discovered. We had no cho ice but to roll up our sleeves to help our community before things got messier than they already were.

As a result, we have released a scenario that will help you detect and block exploitation attempts of the vulnerability. This new scenario can be directly downl oaded from our Hub and installed in a blink of an eye. Check this quick video to see the plugin in action:

https://www.crowdsec.net/blog/detect-block-log4j-exploitation-attempts

- Log4j is a popular software library in the Java world
- CrowdSec quickly provided a list of systems scanning the internet for this vulnerability
  https://gist.github.com/blotus/f87ed46718bfdc634c9081110d243166
- Full disclosure they gave me a hoodie at an event ☺

CrowdSec with NGINX



- **CrowdSec** is an open-source, community-driven security engine – the llama company https://crowdsec.net
- **NGINX Bouncer** – a Lua plugin enforcing ban decisions in real time via `access_by_lua_block`, blocking malicious IPs before requests reach the backend
- **Security Engine** – reads NGINX logs and detects behavioural patterns: HTTP probing, scanning, brute force, and matches against the community blocklist
- **AppSec/WAF component** – inline virtual patching, SQLi, XSS, and known CVE protection; install with: `cscli collections install crowdsecurity/appsec-virtual-patching`
- **Community Blocklist** – crowd-sourced IP reputation feed, continuously updated from millions of sensors worldwide; free tier available
- Two complementary modes: reactive (log-based, bans offenders) and inline (WAF, blocks the request before it is processed)

Source: Docs: https://docs.crowdsec.net/u/bouncers/nginx/

The OWASP Core Rule Set for WAF



- **OWASP Core Rule Set (CRS)** – the standard open source rules, protects against SQL injection, XSS, remote code execution and other common attacks https://coreruleset.org
- CRS uses **anomaly scoring** – each matching rule adds points, the request is blocked when the total passes a threshold. **Paranoia levels** 1–4 – higher levels catch more attacks, but also block more legitimate traffic, start at level 1
- Expect tuning: false positives are normal, run in detection-only mode first and read the logs
- **ModSecurity** – open source Web Application Firewall engine, inspects each HTTP request and response against rules. Works with Apache, NGINX and IIS, version 3 (libmodsecurity) uses a connector module for NGINX
- Trustwave ended support in July 2024, ModSecurity is now an OWASP project https://owasp.org/www-project-modsecurity/
- **Coraza** – newer OWASP WAF engine written in Go, first stable release 2021, fully compatible with CRS v4 https://coraza.io

Image hygiene



- Small base images – fewer packages, fewer CVEs, less for an attacker to use
  e.g. slim, Alpine, distroless – no shell is a feature
- Multi-stage builds – compilers and build tools stay out of the runtime image
- Run as a named non-root user – `USER app` in the Dockerfile
- Pin base images by digest, not only by tag – tags can move
- No secrets in images – layers keep history, `docker history` shows it
- Scan images – e.g. Syft and Grype, or Trivy
- Rebuild regularly – base images get security updates, your image does not

Keeping Container Images secure



- 🔧 `anchore` open-source project that provides a centralized service for inspection, analysis, and certification of container images https://github.com/anchore/anchore-engine
  "As of 2023, Anchore Engine is no longer maintained. There will be no future versions released. Users are advised to use Syft and Grype."
- 🔧 `Syft` https://github.com/anchore/syft SBOM generation and 🔧 `Grype` https://github.com/anchore/grype vulnerability scanning – alternatives to Trivy
- Allow direct download from the internet into your cluster, may become a problem
  – use your own registry or a pull-through proxy
- Malicious people are typosquatting popular containers!
- Supply chain attacks in general are a problem

Inventory and Control of Software Assets



CIS Control 2:
Inventory and Control of Software Assets
Actively manage (inventory, track, and correct) all software on the network so that only authorized software is installed and can execute, and that all unauthorized and unmanaged software is found and prevented from installation or execution.

What versions of software do we depend on?

What software needs to be phased out, upgraded?

SBOM – Software Bill of Materials, a machine-readable list of components
formats CycloneDX and SPDX, generated by tools like syft and trivy

EU Cyber Resilience Act – manufacturers must keep an SBOM

Source: Center for Internet Security CIS Controls 7.1 CIS-Controls-Version-7-1.pdf

Continuous Vulnerability Management



CIS Control 3:
Continuous Vulnerability Management
Continuously acquire, assess, and take action on new information in order to identify vulnerabilities, remediate, and minimize the window of opportunity for attackers.

Acquire – CVE feeds, vendor advisories, OSV, scanners in CI and registry

Assess – not all critical CVEs are critical for you: reachable? exposed? exploited?
CISA Known Exploited Vulnerabilities (KEV) catalog helps prioritize

Act – patch, upgrade, mitigate, or accept the risk with a documented reason

Continuously – an image that was clean last month is not clean today

Source: Center for Internet Security CIS Controls 7.1 CIS-Controls-Version-7-1.pdf

OSV – vulnerabilities and malicious packages



- OSV – Open Source Vulnerabilities, a database and a format, covering many ecosystems
  npm, PyPI, Maven, Go, crates.io, Debian, ...
- Vulnerabilities have IDs like `CVE-` and `GHSA-`
- Malicious packages have IDs starting with `MAL-`
  from the OpenSSF Malicious Packages repository
- `osv-scanner` – scans lockfiles, SBOMs and directories against OSV
- `npm audit` – uses the GitHub Advisory Database

https://osv.dev/

https://github.com/ossf/malicious-packages

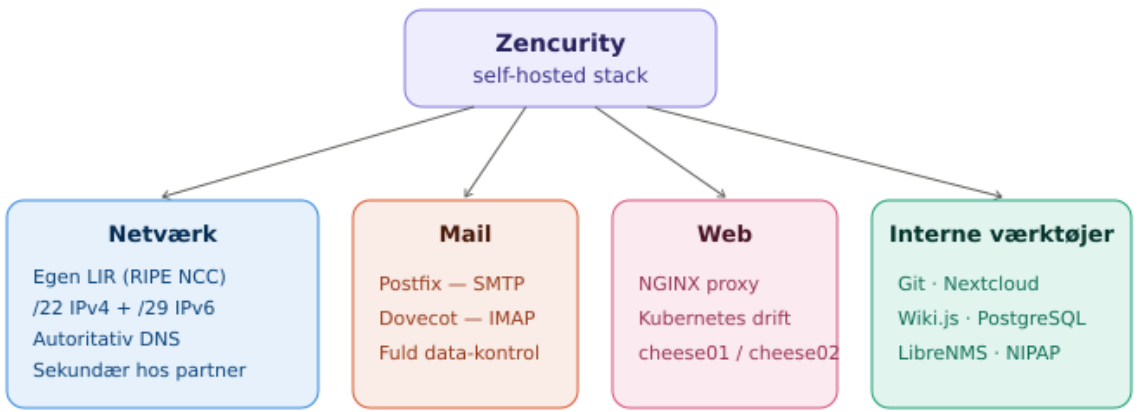
https://google.github.io/osv-scanner/

Overview virtualization



Example servers with KVM and Mixed VM and Kubernetes

Overview services



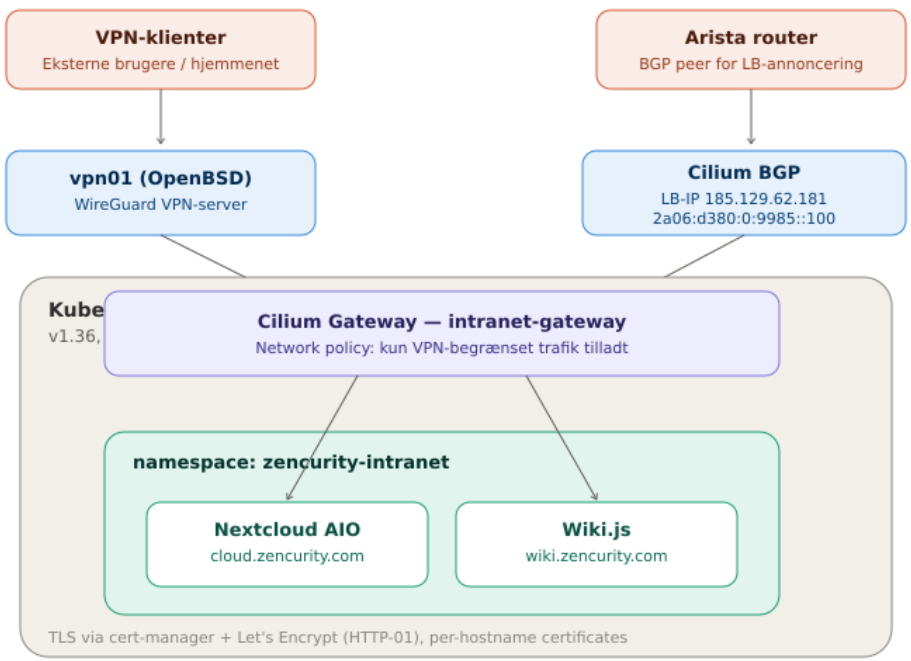
Fælles for det hele

Version-kontrolleret og automatiseret — infrastructure as code fra hypervisor til applikationslag

Self-hosting: vi ejer data, styrer stacken, og er selv på vagt — et bevidst valg

Guest-OS: Debian (workloads) og OpenBSD (sikkerheds- og netværksvendte tjenester)

Kubernetes setup only VPN access to Nextcloud



I do not use all Nextcloud features, so it is only reachable through WireGuard VPN

Real World: Upgrading Nextcloud on Kubernetes



Critical PHP security advisory – PHP 8.3 to 8.4, Nextcloud 33 to 34, about 30 minutes of work:

- Behind a VPN there is no rush – I could upgrade the next day at a good time, not late in the evening
- Run applications in isolated rooms: own Kubernetes namespace, strict NetworkPolicy with both ingress and egress filters
- Full backup first: a script dumping the 3 databases, and tar of all Nextcloud files
- Check the kernel log with `journalctl -k` for PHP segfaults – nothing found
- Look for new PHP files from the last week – not a guarantee, but cheap to run
- Fetch the latest Helm chart for Nextcloud All-in-One, diff the settings for new required values
- Start the upgrade and wait
- The fulltextsearch app had a missing import, a one-line hotpatch https://github.com/nextcloud/fulltextsearch/issues/1005
- Wait a bit more, done

```
helm upgrade nextcloud-aio nextcloud-aio/nextcloud-aio-helm-chart --version 14.4.1
```

All Software Has Errors – Know What You Run



- Every package, library and container image will get vulnerabilities – also the ones you forgot you installed
- Scanning tells you what to fix, patching does the fixing – you need both
- Remove what you do not use – fewer packages, fewer findings
- **SBOM** – Software Bill of Materials, a list of everything installed on a system or in an image
- Example tools: **Grype** matches packages against known vulnerabilities, **Syft** builds SBOMs – both open source from Anchore
  https://github.com/anchore/grype
- Debian hosts: the list of installed apt packages goes straight into Grype
- Kubernetes: Syft builds an SBOM for each container image, then Grype scans the SBOM
  The image is only downloaded once per version

Example: Scanning My Own Infrastructure



- Debian and KVM hosts: a scanner VM collects package lists from 19 hosts over SSH, using a restricted user that can only run a read-only export
- Grype runs nightly, only High/Critical with a fix available
- Push notification with ntfy, only when something new shows up
- First run: 8367 findings – after removing old leftover packages: 827
- 37 Kubernetes images: 1274 findings, nginx:alpine went from 117 to 3 just by pulling the current image
- Floating tags like `latest` bring fixes but change without review – pinned versions are controlled but go stale
- Do not take a new version the minute it is published – most malicious versions are found within days

Installed Is Not the Same as Running



- The scanner sees what is **installed** – a patched kernel package looks clean, but the old kernel is still **running** until you reboot
- Simple check: compare the running kernel with the newest installed one
- If they differ, the host needs a reboot – the nightly scan sends a notification
- Same problem for libraries: a running service keeps using the old version until it is restarted, `needrestart` on Debian can show which
- Kubernetes nodes: kured reboots nodes one at a time, inside a reboot window

The dpkg log shows the install time, and the last boot shows when the new kernel actually started running:

```
for h in cheese01 cheese02 gouda01; do
  echo "== $h"
  ssh $h 'zgrep -h " install linux-image-6.12.113" /var/log/dpkg.log*; uptime -s'
done
```

Break

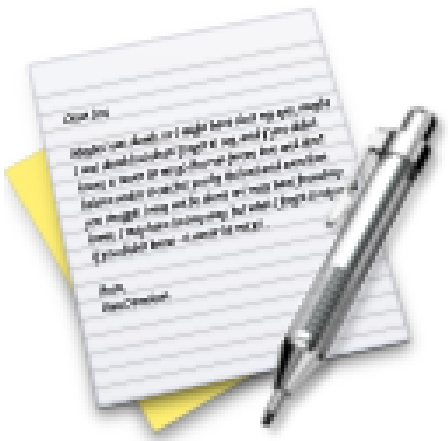


Photo by Alexas Fotos on Unsplash

Session 1: Exercises: PMD Static Analysis and SBOM Scanning



Exercise

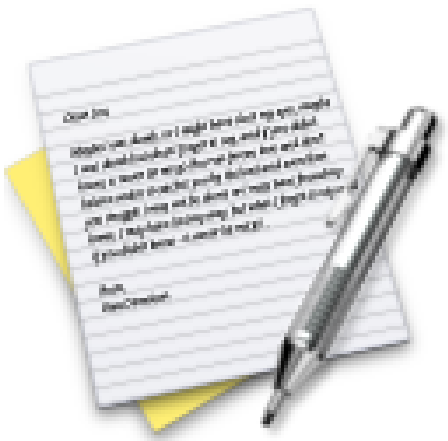


Now lets do the exercise

⚠ Trying PMD static analysis 30 min

which is number **29** in the exercise PDF.

Exercise



Now lets do the exercise

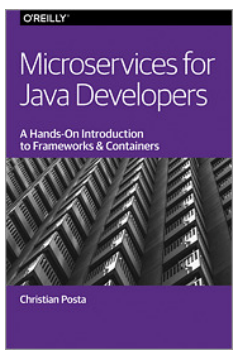
⚠ SBOM and vulnerability scan of Juice Shop with Syft and Grype up to 45 min

which is number 32 in the exercise PDF.

Session 2: Teaching: Sample Infrastructure, API Versioning and Containers



Microservices for Java chapter 1



Microservices for Java Developers , Christian Posta, 2016 O'Reilly
https://www.oreilly.com/programming/free/files/microservices-for-java-developers.pdf

We will use the introduction, which is recommended reading.

Whats in the book



> This book is for Java developers and architects interested in developing microservices. We start the book with the high-level understanding and fundamental prerequisites that should be in place to be successful with a microservice architecture.

Introducing some Java frameworks

- The Spring ecosystem, Dropwizard and WildFly Swarm, we'll use JBoss Forge CLI
- Finally, when we build and deploy our microservices as Docker containers running inside of Kubernetes
- They use VirtualBox with Docker and Kubernetes, YMMV
- With source on Github, not updated in years though!
  https://github.com/redhat-developer/microservices-by-example-source

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Open source is also leading the charge in the technology space



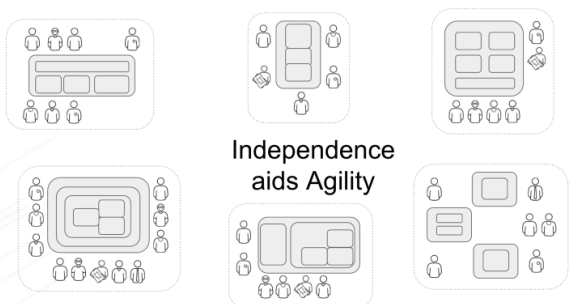
> Open source is also leading the charge in the technology space. Following the commoditization curves, open source is a place developers can go to challenge proprietary vendors by building and innovating on software that was once only available (without source no less) with high license costs. This drives communities to build things like operating systems (Linux), programming languages (Go), message queues (Apache ActiveMQ), and web servers (httpd). Even companies that originally rejected open source are starting to come around by open sourcing their technologies and contributing to existing communities. As open source and open ecosystems have become the norm, we're starting to see a lot of the innovation in software technology coming directly from open source communities (e.g., Apache Spark, Docker, and Kubernetes).

- We have used Linux and multiple products from the Apache website/foundation

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Building distributed systems is hard



Microservice architecture (MSA) is an approach to building software systems that decomposes business domain models into smaller, consistent, bounded-contexts implemented by services. These services are isolated and autonomous yet communicate to provide some piece of business functionality. Microservices are typically implemented and operated by small teams with enough autonomy that each team and service can change its internal implementation details (including replacing it outright!) with minimal impact across the rest of the system.

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Teamwork



- Teams communicate through promises
- specify these promises with interfaces of their services and via wikis that document their services
- Each team would be responsible for designing the service, picking the right technology for the problem set, and deploying, managing and waking up at 2 a.m. for any issues
- Understand what the service is doing without being tangled into other concerns in a larger application
- Quickly build the service locally
- Pick the right technology for the problem (lots of writes? lots of queries? low latency? bursty?)
- Test the service
- Build/deploy/release at a cadence necessary for the business, which may be independent of other services
- Identify and horizontally scale parts of the architecture where needed
- Improve resiliency of the system as a whole

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Challenges



- Microservices may not be efficient. It can be more resource intensive.
- You may end up with what looks like duplication.
- Operational complexity is a lot higher.
- It becomes very difficult to understand the system holistically.
- It becomes significantly harder to debug problems.
- In some areas you may have to relax the notion of transaction.

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Design for Faults



> Things will fail, so we must develop our applications to be resilient and handle failure, not just prevent it. We should be able to deal with faults gracefully and not let faults propagate to total failure of the system.
>
> Building distributed systems is different from building shared- memory, single process, monolithic applications. One glaring differ- ence is that communication over a network is not the same as a local call with shared memory.

- Networks are inherently unreliable

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Design with Dependencies in Mind



> To be able to move fast and be agile from an organization or distributed-systems standpoint, we have to design systems with dependency thinking in mind; we need loose coupling in our teams, in our technology, and our governance.

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Design with Promises in Mind



> In a microservice environment with autonomous teams and serv- ices, it's very important to keep in mind the relationship between service provider and service consumer. As an autonomous service team, you cannot place obligations on other teams and services because you do not own them; they're autonomous by definition. All you can do is choose whether or not to accept their promises of functionality or behavior. As a provider of a service to others, all you can do is promise them a certain behavior.

- Promises as published by APIs and versions in those!
- References *Consumer-Driven Contracts: A Service Evolution Pattern* https://martinfowler.com/articles/consumerDrivenContracts.html

Source:

Microservices for Java Developers , Christian Posta, 2016 O'Reilly

Chapter 10: Service API and Contract Versioning with Web Services and REST Services



After a service contract is deployed, consumer programs will naturally begin forming dependencies on it. When we are subsequently forced to make changes to the contract, we need to figure out:

- Whether the changes will negatively impact existing (and potentially future) service consumers
- How changes that will and will not impact consumers should be implemented and communicated

Included in the chapter are references to:

- Versioning Web Services
- Versioning REST Services
- Fine and Coarse-Grained Constraints

Source:

Service-Oriented Architecture: Analysis and Design for Services and Microservices, Thomas Erl, 2017

Versioning and Compatibility



- Backwards Compatibility
  A backwards-compatible change to a REST-compliant service contract might involve adding some new resources or adding new capabilities to existing resources. In each of these cases the existing service consumers will only invoke the old methods on the old resources, which continue to work as they previously did.
- Forwards Compatibility
  When a service contract is designed in such a manner so that it can support a range of future consumer programs, it is considered to have an extent of forwards compatibility. This means that the contract can essentially accommodate how consumer programs will evolve over time.

Source:

Service-Oriented Architecture: Analysis and Design for Services and Microservices, Thomas Erl, 2017

Kubernetes API Versioning – excerpt



Alpha, e.g. *v1alpha1*

- Disabled by default, must be enabled explicitly
- May contain bugs, can be dropped at any time without notice
- May change incompatibly, only for short-lived test clusters

Beta, e.g. *v2beta3*

- Disabled by default (except betas introduced before 1.22)
- Well tested, the feature will not be dropped, but details may change
- Incompatible changes come with migration instructions
- Max lifetime: 9 months or 3 releases to deprecation, and the same again to removal

Stable, e.g. *v1*

- Available in all future releases within a major version

Source:

API Overview, Kubernetes Documentation, CC BY 4.0 https://kubernetes.io/docs/reference/using-api/

Docker



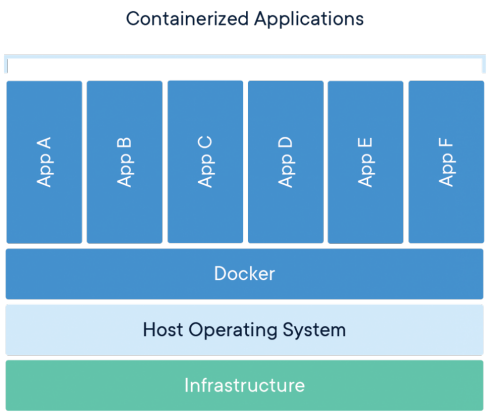
> Package Software into Standardized Units for Development, Shipment and Deployment A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

Source:

https://www.docker.com/resources/what-container

- One of the most popular deployment methods today

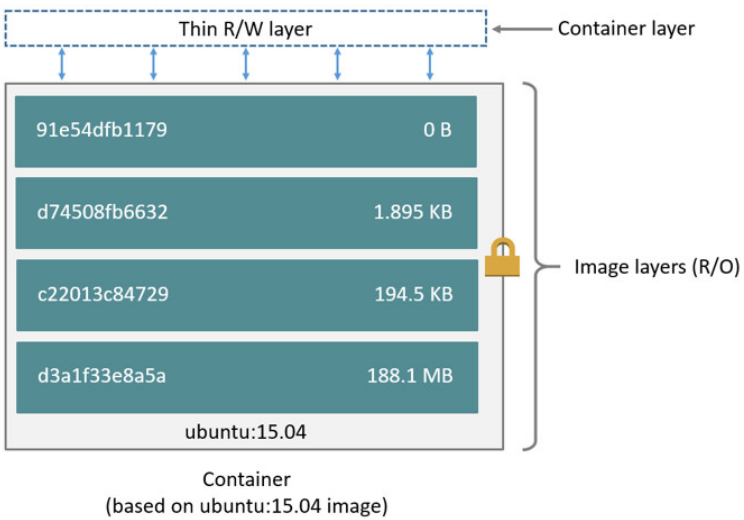
Containerized Applications



Source: https://www.docker.com/resources/what-container

- See also https://en.wikipedia.org/wiki/Linux_namespaces
  Various container software use Linux namespaces in combination with cgroups to isolate their processes, including Docker[11] and LXC.

Docker Images and layers



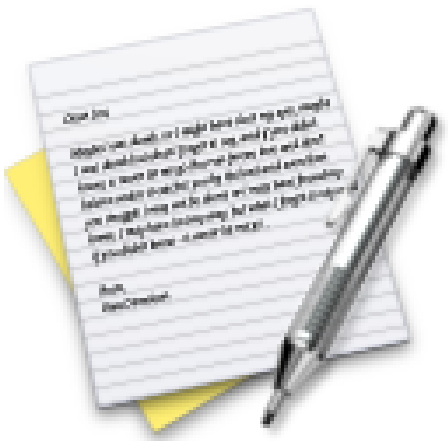
A Docker image is built up from a series of layers. Each layer represents an instruction in the image's Dockerfile. Each layer except the very last one is read-only.

Source: https://docs.docker.com/storage/storagedriver/

Session 3: Exercises: XML Library in Python



Exercise

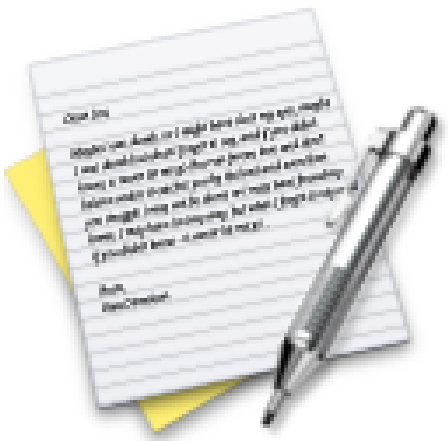


Now lets do the exercise

ℹ Use a XML library in Python up to 45min

which is number **31** in the exercise PDF.

Exercise

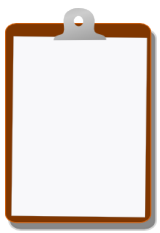


Now lets do the exercise

ℹ Whatweb Scanner 15 min

which is number **18** in the exercise PDF.

For Next Time



Think about the subjects from this time, write down questions

Check the plan for chapters to read in the books

Visit web sites and download papers if needed

Retry the exercises to get more confident using the tools